Linked List – continued

So far, we have said that the list is a data structure which has elements identifiable by ordinal positions. It has two properties.

1. Access to an element is sequential – no direct access.
2. Insertion and deletion once the position is known, requires a few operations. It does not depend on the number of elements in the list.

We have seen that the list can be implemented as a linked list.

We have also created an ordered list where the nodes in the list are arranged based on a key field. We have also discovered that the insert operation has 4 different boundary conditions.

Can we change the implementation without affecting the client? Why not? A data structure is an ADT – abstract data type. It specifies the interface through the header file in 'C'. If we change the implementation file without changing the header file, then the client code need not be even recompiled.
Note: In certain cases, in C and C++, the client code need to be recompiled without changing the code.

We will compile our client code and keep it separate. Please note the time stamp by using the dir or ls -l command.

We will change the concept of our list. We will say that our empty list will have one node. So, while initializing the list, we will create one dummy node – called the header node. In our example, we may not initialize the key_ field. Of course, we will ground the next_ field.

```
void init_list(mylist_t* ptr_list)
{

        ptr_list->head_ = (node_t*)malloc(sizeof(node_t));
        ptr_list->head_->next_ = NULL;
}
```

Since the list has a dummy header node, we may have to change all other functions.

The display function should skip one node and then start displaying.

```
void disp_list(mylist_t* ptr_list)

{

        disp_(ptr_list->head_->next_);
}
```

The function free list goes back to the beginning, ready to build the list again. The list has one dummy node with its next_ field grounded.

What is left? Insert function. Let us examine the function. We said earlier that there are four boundary conditions which should take care.

1. Empty list : where the head_ which is NULL has to be changed. But in our case, head_ is never NULL. So, this clause is not even necessary.
2. Insert in the beginning of the list requiring head_ to be changed. This also will not happen.
3. & 4. Insert in the middle or the end. Let us examine as we go by.

Let us examine the code for insertion.

```
void insert_list(mylist_t* ptr_list, int key)
{
        // create the node
        node_t* temp = (node_t*)malloc(sizeof(node_t));
        temp->key_ = key; temp->next_ = NULL;

        // This goes!!
        // 1.empty list
        if(ptr_list->head_ == NULL)
        {
                ptr_list->head_ = temp;
                temp->next_ = NULL;
        }
```

```
        Else // NO else
        {
                // traverse // requires changes in initialization
                node_t* prev = NULL;
                node_t* pres = ptr_list->head_;

                while(pres != NULL && pres->key_ < temp->key_ )
                {
                        prev = pres;
                        pres = pres->next_;
                }
                // this also goes
                if(prev == NULL) // insert in the beginning
                {
                        temp->next_ = pres;
                        ptr_list->head_ = temp;
                }
                else // middle or end
                {
                        temp->next_ = pres;
                        prev->next_ = temp;
                }

        }


}
```

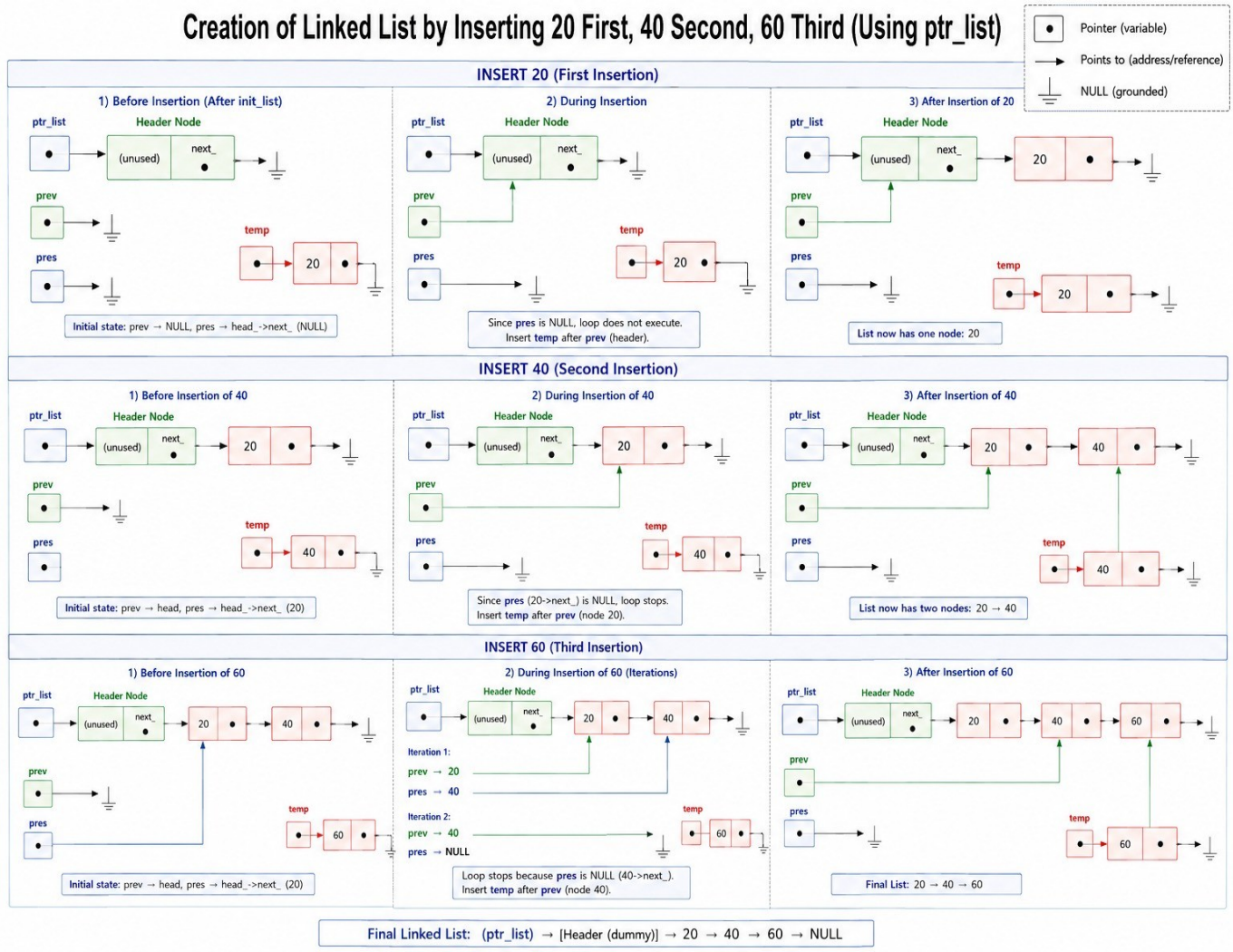
Now the changed function looks like this.

```
// modified code
void insert_list(mylist_t* ptr_list, int key)
{
        // create the node
        node_t* temp = (node_t*)malloc(sizeof(node_t));
        temp->key_ = key; temp->next_ = NULL;

        // traverse
        // observe the changes here
        node_t* prev = ptr_list->head_;
        node_t* pres = ptr_list->head_->next_;


        while(pres != NULL && pres->key_ < temp->key_ )
        {
                prev = pres;
                pres = pres->next_;
        }

        temp->next_ = pres;
        prev->next_ = temp;
}
```



This looks so simple. Is this not easy to read? Our insertion function has become extremely readable.

So, we use the header node to make insertion simple.

The client is not affected. We can recompile this new code, link with the object file of the client and run the code.

Similarly can we have one more node at the end – tail or tailer node? If yes, what could be the reason for it?

// empty list : diagram
// insert into this list : no change in head : diagram

```
// insert in the middle
// insert at the beginning
// insert at the end
```